

```

10 merged = []
11 i = 0
12 j = 0
13 count = c1 + c2
14
15 while i < len(left) and j < len(right):
16     count += 1
17
18     if left[i] <= right[j]:
19         merged.append(left[i])
20         i += 1
21     else:
22         merged.append(right[j])
23         j += 1
24
25 merged += left[i:]
26 merged += right[j:]
27
28 return merged, count
29
30
31 n = int(input())
32 arr = list(map(int, input().split()))
33
34 sorted_arr, comparisons = merge_sort(arr)
35
36 print(*sorted_arr, sep=",")
37 print("Comparisons :", comparisons)

```

input

```

5
14 15 78 95 16
14,15,16,78,95
Comparisons : 6

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

4      if low < high:
5          pivot = arr[high]
6          i = low - 1
7
8          for j in range(low, high):
9              comparisons += 1
10
11             if arr[j] <= pivot:
12                 i += 1
13                 arr[i], arr[j] = arr[j], arr[i]
14
15             arr[i + 1], arr[high] = arr[high], arr[i + 1]
16
17             p = i + 1
18
19             quick_sort(arr, low, p - 1)
20             quick_sort(arr, p + 1, high)
21
22
23 n = int(input())
24 arr = list(map(int, input().split()))
25
26 comparisons = 0
27
28 quick_sort(arr, 0, n - 1)
29
30 print(*arr, sep=",")
31 print("Comparisons :", comparisons)

```

```

6
15 18 98 78 86 35
15,18,35,78,86,98
Comparisons : 9

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

37
38     for j in range(low, high):
39         quick_count += 1
40
41         if arr[j] <= pivot:
42             i += 1
43             arr[i], arr[j] = arr[j], arr[i]
44
45     arr[i + 1], arr[high] = arr[high], arr[i + 1]
46
47     p = i + 1
48
49     quick_sort(arr, low, p - 1)
50     quick_sort(arr, p + 1, high)
51
52
53 n = int(input())
54 arr = list(map(int, input().split()))
55
56 merge_arr, merge_count = merge_sort(arr.copy())
57
58 quick_arr = arr.copy()
59 quick_count = 0
60 quick_sort(quick_arr, 0, n - 1)
61
62 print("Sorted Array :", *merge_arr, sep=" ")
63 print("Merge Comparisons :", merge_count)
64 print("Quick Comparisons :", quick_count)

```


inp

```

6
12 11 10 5 6 7
Sorted Array : 5 6 7 10 11 12
Merge Comparisons : 8
Quick Comparisons : 9

```

...Program finished with exit code 0

```

13     i = 0
14     j = 0
15
16     while i < len(left) and j < len(right):
17         if left[i] <= right[j]:
18             result.append(left[i])
19             i += 1
20         else:
21             result.append(right[j])
22             j += 1
23
24     result += left[i:]
25     result += right[j:]
26
27     return result
28
29
30 n = int(input())
31 arr = list(map(int, input().split()))
32
33 start = time.time()
34
35 sorted_arr = merge_sort(arr)
36
37 end = time.time()
38
39 print(*sorted_arr, sep=",")
40 print("Time Taken :", end - start, "sec")

```



```

5
4 6 5 7 8
4,5,6,7,8
Time Taken : 3.314018249511719e-05 sec

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

4     if low < high:
5         pivot = arr[high]
6         i = low - 1
7
8         for j in range(low, high):
9             comparisons += 1
10
11             if arr[j] <= pivot:
12                 i += 1
13                 arr[i], arr[j] = arr[j], arr[i]
14
15         arr[i + 1], arr[high] = arr[high], arr[i + 1]
16
17         p = i + 1
18
19         quick_sort(arr, low, p - 1)
20         quick_sort(arr, p + 1, high)
21
22
23 n = int(input())
24 arr = list(map(int, input().split()))
25
26 comparisons = 0
27
28 quick_sort(arr, 0, n - 1)
29
30 print(*arr, sep=",")
31 print("Comparisons :", comparisons)

```

input

```

5
14 54 78 65 95
14,54,65,78,95
Comparisons : 8

```

...Program finished with exit code 0
Press ENTER to exit console.

```

20
21     left = merge_sort(arr[:mid], threshold)
22     right = merge_sort(arr[mid:], threshold)
23
24     result = []
25     i = 0
26     j = 0
27
28     while i < len(left) and j < len(right):
29         if left[i] <= right[j]:
30             result.append(left[i])
31             i += 1
32         else:
33             result.append(right[j])
34             j += 1
35
36     result += left[i:]
37     result += right[j:]
38
39     return result
40
41
42 n = int(input())
43 arr = list(map(int, input().split()))
44
45 result = merge_sort(arr)
46
47 print(*result, sep=",")

```



```

5
65 89 75 15 25
15,25,65,75,89

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

10 merged = []
11 i = 0
12 j = 0
13 count = c1 + c2
14
15 while i < len(left) and j < len(right):
16
17     if left[i] <= right[j]:
18         merged.append(left[i])
19         i += 1
20     else:
21         merged.append(right[j])
22         j += 1
23         count += len(left) - i
24
25 merged += left[i:]
26 merged += right[j:]
27
28 return merged, count
29
30
31 n = int(input())
32 arr = list(map(int, input().split()))
33
34 sorted_arr, inversions = merge_sort(arr)
35
36 print("Sorted :", *sorted_arr, sep=" ")
37 print("Inversions :", inversions)

```



```

5
2 1 4 3 5
Sorted : 1 2 3 4 5
Inversions : 2

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

5
6     if low < high:
7         if depth > max_depth:
8             max_depth = depth
9
10        pivot = arr[high]
11        i = low - 1
12
13        for j in range(low, high):
14            if arr[j] <= pivot:
15                i += 1
16                arr[i], arr[j] = arr[j], arr[i]
17
18        arr[i + 1], arr[high] = arr[high], arr[i + 1]
19
20        p = i + 1
21
22        quick_sort(arr, low, p - 1, depth + 1)
23        quick_sort(arr, p + 1, high, depth + 1)
24
25
26 n = int(input())
27 arr = list(map(int, input().split()))
28
29 quick_sort(arr, 0, n - 1, 1)
30
31 print(*arr, sep=",")
32 print("Max Depth :", max_depth)

```



```

5
14 54 74 25 35
14,25,35,54,74
Max Depth : 2

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```
1 a = list(map(int, input().split()))
2 b = list(map(int, input().split()))
3
4 i = 0
5 j = 0
6 result = []
7
8 while i < len(a) and j < len(b):
9     if a[i] <= b[j]:
10         result.append(a[i])
11         i += 1
12     else:
13         result.append(b[j])
14         j += 1
15
16 while i < len(a):
17     result.append(a[i])
18     i += 1
19
20 while j < len(b):
21     result.append(b[j])
22     j += 1
23
24 print(*result, sep=",")
```



```
4 5 6
1 2 3
1,2,3,4,5,6
```

...Program finished with exit code 0
Press ENTER to exit console.

```

29     if low < high:
30         pivot = arr[high]
31         i = low - 1
32
33         for j in range(low, high):
34             if arr[j] <= pivot:
35                 i += 1
36                 arr[i], arr[j] = arr[j], arr[i]
37
38         arr[i + 1], arr[high] = arr[high], arr[i + 1]
39
40         p = i + 1
41
42         quick_sort(arr, low, p - 1)
43         quick_sort(arr, p + 1, high)
44
45
46 n = int(input())
47 arr = list(map(int, input().split()))
48
49 merge_result = merge_sort(arr.copy())
50
51 quick_result = arr.copy()
52 quick_sort(quick_result, 0, n - 1)
53
54 print("Sorted :", *merge_result, sep=" ")
55 print("Merge Sort uses extra array space.")
56 print("Quick Sort uses mainly recursion stack space.")

```



```

5
5 4 2 1 3
Sorted : 1 2 3 4 5
Merge Sort uses extra array space.
Quick Sort uses mainly recursion stack space.

...Program finished with exit code 0

```

```

8         arr[i], arr[j] = arr[j], arr[i]
9
10    arr[i + 1], arr[high] = arr[high], arr[i + 1]
11
12    return i + 1
13
14
15    def kth_smallest(arr, low, high, k):
16        if low <= high:
17            p = partition(arr, low, high)
18
19            if p == k - 1:
20                return arr[p]
21
22            elif p > k - 1:
23                return kth_smallest(arr, low, p - 1, k)
24
25            else:
26                return kth_smallest(arr, p + 1, high, k)
27
28
29    n = int(input())
30    arr = list(map(int, input().split()))
31    k = int(input())
32
33    answer = kth_smallest(arr, 0, n - 1, k)
34
35    print(answer)

```

input

```

5
7 6 5 4 1
5
7

```

Program finished with exit code 0

```

36     i = low - 1
37
38     for j in range(low, high):
39         quick_count += 1
40
41         if arr[j] <= pivot:
42             i += 1
43             arr[i], arr[j] = arr[j], arr[i]
44
45     arr[i + 1], arr[high] = arr[high], arr[i + 1]
46
47     p = i + 1
48
49     quick_sort(arr, low, p - 1)
50     quick_sort(arr, p + 1, high)
51
52
53 n = int(input())
54 arr = list(map(int, input().split()))
55
56 _, merge_count = merge_sort(arr.copy())
57
58 quick_count = 0
59 quick_arr = arr.copy()
60 quick_sort(quick_arr, 0, n - 1)
61
62 print("Merge Comparisons :", merge_count)
63 print("Quick Comparisons :", quick_count)

```

input

```

5
17 1 6 2 4
Merge Comparisons : 8
Quick Comparisons : 6

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```